

LOWER-LIMB JOINT ANGLE PREDICTIONS USING CNN-BiLSTM MODEL

2/5/2026

Grace Kim,
Mechanical and Mechatronics Engineering

Project Overview

Final Goal of the Project:

- Develop a neural network model to predict joint angle trajectories for one future gait cycle, given previous gait data

Objectives:

- Preprocessing data
- Design a neural network model
- Train the model to generate joint angle predictions



Model Inputs and Outputs

Input: 35 channels

- 5 channels: EMGs
- 18 channels: Leg IMUs (3 sensors x (3-axis acceleration + 3-axis gyroscope))
- 6 channels: Center chest IMU (3-axis acceleration + 3-axis gyroscope)
- 6 channels: Lower back IMU (3-axis acceleration + 3-axis gyroscope)

Output: 5 channels

- Hip joint angles: X, Y, Z
- Knee joint angle: PCA-reduced component
- Ankle joint angle: PCA-reduced component

RESHAPING DATA

PCA → gait segmentation → sliding window batch creation → dataset splitting

Principal Component Analysis (PCA)

1. Global PCA Fitting

- Stack knee and ankle angle samples across all files separately
- Fit PCA with $n = 1$ for each joint (imported from scikit-learn)
- Save PCA models for knee and ankle

2. PCA Application to each File

- Transform 3-axis joint angles into 1D PCA value
- 1D regression targets instead of predicting all 3 axes

Final model output targets: [Hip X, Hip Y, Hip Z, Knee PCA, Ankle PCA]

Gait Cycle Segmentation

Continuous walking data with HS_bool column



Heel strike detection ($0 \rightarrow 1$) to segment individual gait cycles



Time normalization to gait cycle percentage (0-100%)



Resample each cycle to 200 points



Trim first & last 'k' cycles

Sliding Window Batch Generation

Objective:

1. Input window creation – use n consecutive gait cycles as input
2. output window creation – predict the $(n+1)^{\text{th}}$ gait cycle

Example ($n = 10$):

Input cycles	Prediction Target
Cycles 1 – 10	Cycle 11
Cycles 2 – 11	Cycle 12
Cycles 3 – 12	Cycle 13

Slide the window forward by one cycle to generate training batches.

Train/Validation/Test Data Splitting

1. Randomly shuffle the batches within each file
2. Split dataset per file:
 - Training set (70%) – used for model learning
 - Validation set (15%) – used for hyperparameter tuning and monitors overfitting during training
 - Test set (15%) – final performance evaluation
3. Each of the sets are combined for all the files

This approach prevents biases and preserves participant variability

NEURAL NETWORK DESIGN

Model Shape Overview

Input: previous gait data

- Shape: $(200 \cdot n, 35, 1)$
 - 200 points per gait cycle
 - n cycles to be used as previous data ($n = 10$ for this project)
 - 35 sensor/feature channels

Output: predicted next gait cycle joint angles

- Shape: $(200, \text{OUT}) = (200, 5)$
 - Targets: [HipX, HipY, HipZ, KneePCA, AnklePCA]

Architecture Overview



1. Conv2D temporal feature extraction (ELU activation) with decreasing kernel sizes $(7, 1) \rightarrow (5, 1) \rightarrow (3, 1)$
2. Reshape for LSTM & reduce feature dimension using a dense layer
3. Average pooling to reduce sequence length
4. BiLSTM processes the entire gait sequence
5. Dense & reshape for output. Linear activation is used.

Design Rationale and Training Configuration

Why CNN-BiLSTM?

- CNN layers extract short-term local temporal patterns
- BiLSTM captures long-range temporal dependencies across multiple gait cycles

Training Configuration:

- Optimizer: Adam (LR = $1e-3$)
- Loss: MSE
- Metric: MAE (training); RMSE (final reporting)
- Epochs: up to 200 (Early Stopping used)
- Batch size: 16
- Callbacks: ModelCheckpoint (best validation loss), EarlyStopping, ReduceLROnPlateau

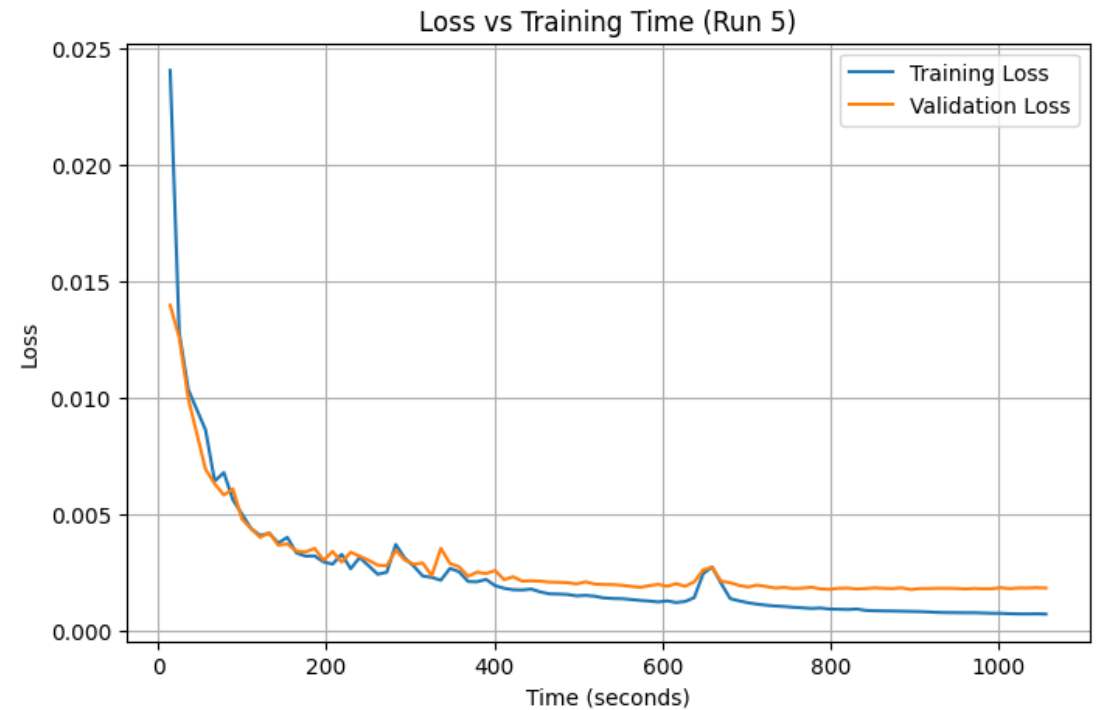
RESULTS

Training the Model

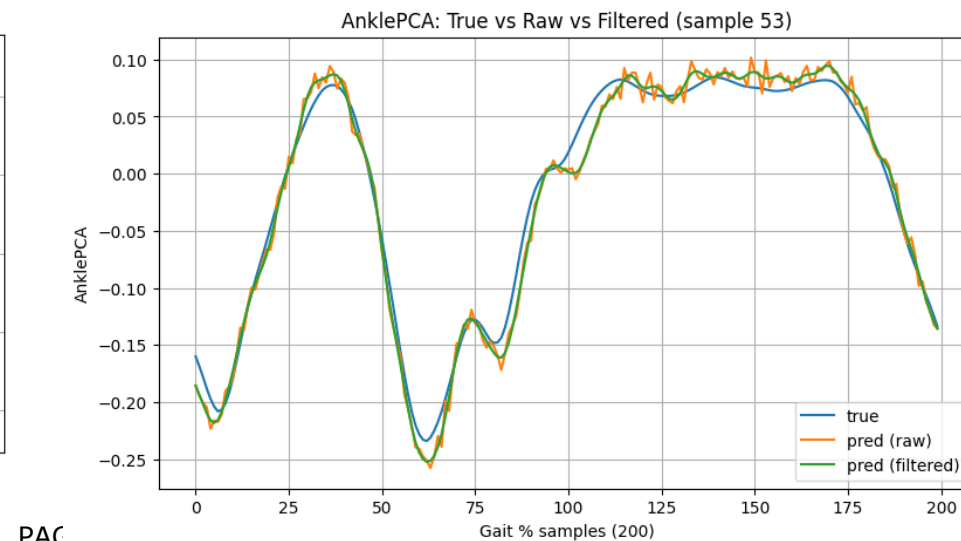
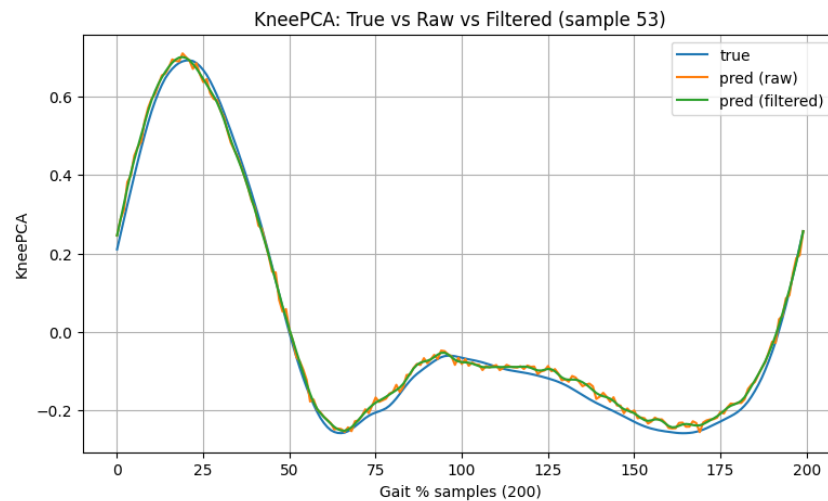
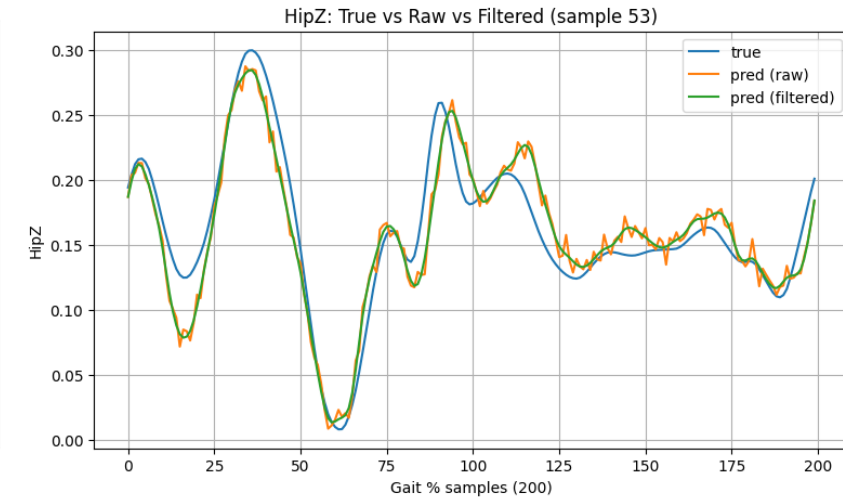
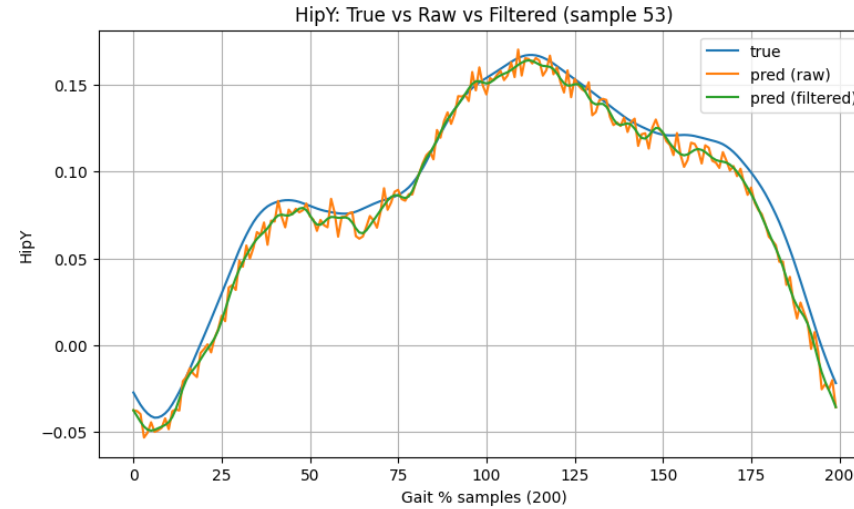
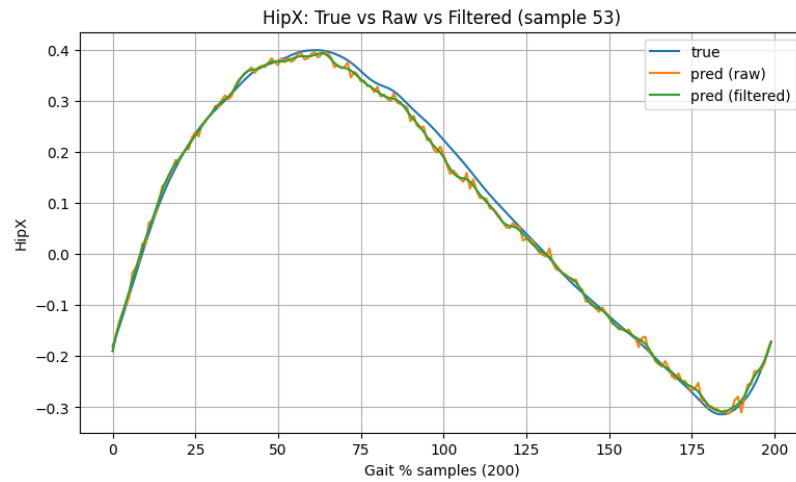
- 5 independent runs
- Test RMSE comparison:

Best run	Worst run
0.0437 rad	0.0482 rad

- Mean test RMSE: 0.0451 ± 0.0018 rad ($\sim 2.58^\circ$)
- Mean training time: 1010.35 s (~ 17 min)



Trajectory Comparison



RMSE Values per Output

Run	Val_rmse	Test_rmse	Train time (s)	HipX	HipY	HipZ	KneePCA	AnklePCA
1	0.03912	0.04385	974.50	0.04250	0.01793	0.04090	0.06313	0.04277
2	0.03772	0.04469	905.73	0.04645	0.01840	0.04087	0.06260	0.04360
3	0.03869	0.04526	1054.02	0.04553	0.01855	0.04155	0.06436	0.04423
4	0.03881	0.04823	1060.94	0.05188	0.01919	0.04202	0.06858	0.04582
5	0.03901	0.04371	1056.54	0.04282	0.01786	0.04146	0.06136	0.04376
Mean	0.03867	0.04515	1015.35	0.04584	0.01834	0.04136	0.06401	0.04404

- HipY shows the lowest prediction errors, while KneePCA shows the highest errors
- Knee angles have a lot of changes in amplitudes during walking

CONCLUSION

Limitations & Next Steps

- Optimizing the model architecture
- Perform systematic hyperparameter tuning
- Training on larger and more diverse datasets
- Exploring alternative neural network architectures

**UNIVERSITY OF
WATERLOO**



FACULTY OF ENGINEERING